

Introducción a JavaScript:

Tipo de dato

- Number
- String

Variables en JavaScript

- let
- var
- const

Consola de JavaScript

Operadores aritméticos, operadores lógicos y unarios.

Puente Cognitivo

Las páginas web que construimos con HTML y CSS son estáticas; son como un documento de Word con un diseño atractivo. El objetivo de hoy es introducir el "motor" que le da vida a esa estructura. Pasaremos de estructurar contenido visual a procesar datos invisibles en la memoria del navegador.

Reto: Analicen el siguiente fragmento de código HTML. Su tarea no es corregirlo, sino identificar qué elemento visual requiere, por obligación, una operación matemática o lógica por detrás para funcionar en el mundo real.

```
<div class="product-card">  
  <h2>Camiseta Básica</h2>  
  <p class="price">$20.00</p>  
  <button class="btn-add">Agregar al carrito</button>  
  <p class="stock-alert">¡Solo quedan 2 unidades!</p>  
</div>
```

Situación

Imagina que eres el desarrollador de una plataforma como MercadoLibre o Amazon. Un usuario selecciona tres productos diferentes. HTML puede mostrar las fotos y CSS puede ponerlas en una cuadrícula perfecta, pero ninguno de los dos lenguajes sabe sumar.

Ninguno puede calcular el IVA del 19%, ni aplicar un cupón de descuento del 10%, ni mostrar un mensaje de bienvenida que diga "Hola, Juan".

Para transformar una maqueta inerte en una aplicación interactiva, necesitamos **JavaScript**. Y el primer paso para dominar JavaScript es aprender a guardar información (variables), definir qué tipo de información es (números o texto) y operarla matemáticamente o lógicamente.

JavaScript es el "cerebro" detrás de esa aplicación.

Variables

La programación se trata fundamentalmente de procesar datos. Para procesar datos, primero debemos almacenarlos en la memoria de la computadora.

Una variable es un contenedor con un nombre donde almacenamos un valor para usarlo más tarde. JavaScript es un lenguaje de **tipado dinámico**, lo que significa que no necesitas decirle a la variable qué tipo de dato va a contener (a diferencia de Java o C#).

- **let (La Caja Estándar):** Es la forma moderna y preferida de declarar una variable cuyo valor **puede cambiar**. Solo vive dentro de las llaves { ... } donde fue creada.
- **const (La Caja Fuerte):** Declara una variable cuyo valor **no puede ser reasignado**. Es una "constante". Se debe usar por defecto, a menos que sepas que el valor cambiará.
- **var (La Caja Vieja y "Con Fugas"):** Es la forma antigua de declarar variables. Tiene un comportamiento problemático con el *alcance (scope)*, lo que puede causar errores difíciles de rastrear. **Práctica moderna: Evitar var por completo.**

var vs. let (Scope y Hoisting)

Este es el "por qué" evitamos var.

- **Scope (Alcance):** Define dónde es visible una variable.
 - var tiene **alcance de función**. Si la declaras dentro de un bloque {} (como un if o un for), ¡sigue existiendo fuera de ese bloque! Esto es peligroso.
 - let y const tienen **alcance de bloque**. Solo existen dentro del bloque {} donde fueron declaradas. Esto es predecible y seguro.
- **Hoisting (Elevación):** JavaScript "eleva" las declaraciones.
 - var es elevado y se inicializa con undefined. Puedes usar la variable *antes* de declararla y su valor será undefined.
 - let y const también son elevadas, pero no se inicializan. Si intentas acceder a ellas antes de su declaración, entras en la "Zona Muerta Temporal" (Temporal Dead Zone - TDZ) y obtienes un ReferenceError. Esto es bueno, ya que te obliga a escribir código más limpio.

Tipos de Datos Fundamentales

- **Number (Números):** JavaScript tiene un solo tipo para todos los números, ya sean enteros (10) o decimales (10.5).
- **String (Cadenas de Texto):** Cualquier cosa entre comillas (") o comillas dobles ("").
- **Boolean:** true (verdadero) o false (falso). Base de toda la lógica computacional.

Valores Especiales:

- **Infinity:** El resultado de $1 / 0$.
- **NaN (Not a Number):** El resultado de una operación matemática inválida, como $0 / 0$ o 'texto' * 2. Es el único valor en JavaScript que no es igual a sí mismo ($\text{NaN} === \text{NaN}$ es false).
- **Concatenación:** Se pueden "sumar" (unir) strings usando el operador +.
- **Template Literals (La Forma Moderna):** Usando backticks (comillas invertidas `), podemos crear strings de manera más limpia e insertar variables directamente. Esto se llama interpolación.

Anatomía del Código

```
let nombre = 'Juan';
```

```
let edad = 25;
```

```
// Forma Antigua (Concatenación)
```

```
let saludoViejo = 'Hola, mi nombre es ' + nombre + ' y tengo ' + edad + ' años.';
```

```
// Forma Moderna (Template Literals) - ¡Mucho mejor!
```

```
let saludoModerno = `Hola, mi nombre es ${nombre} y tengo ${edad} años.`;  
console.log(saludoModerno); // "Hola, mi nombre es Juan y tengo 25 años."
```

Evidencia los tipos con typeof

Recomendaciones para nombrar variables en JavaScript

- Usa nombres descriptivos
- Usa camelCase
- Evita abreviaciones innecesarias
- No uses palabras reservadas
- No empieces con números o símbolos (se permite \$ y _)
- Evita poner el tipo de dato en el nombre
- Usa nombres afirmativos o con sentido lógico para booleanos
- No uses tildes ni eñes
- Recuerda que JavaScript distingue mayúsculas y minúsculas
- Sé coherente y constante con tu estilo
- Usa la misma convención en todo tu proyecto

Consola de JavaScript

El navegador (Chrome, Edge, Firefox) tiene un motor de JavaScript integrado (V8 en el caso de Chrome). La consola es nuestra línea directa con ese motor. Comenzaremos abriendo la herramienta más importante para un desarrollador: la **Consola del Navegador** (F12).

Utilizamos el comando `console.log()` para imprimir valores en la pantalla del desarrollador.

```
console.log("Hola Mundo");
```

```
console.log("Viajando a " + ciudad + ", en " + pais + ".");
```

```
console.log("El producto es:", producto, "y cuesta:", precio, "dólares.");
```

Template Literals

```
console.log(`Mi banda favorita es ${artista} y su mejor canción es ${cancion}.`);
```

Operadores - Los operadores nos permiten manipular nuestras variables.

- **Aritméticos:** (+, -, *, /, %, **). + (suma), - (resta), * (multiplicación), / (división), ** (exponenciación - ES6), % (módulo o residuo de una división).
- **Unarios:** Operan sobre un solo valor.
 - ++ (Incremento): x++ (post-incremento) o ++x (pre-incremento). Aumenta el valor en 1.
 - -- (Decremento): x-- o --x. Disminuye el valor en 1.
 - ! (Negación Lógica - NOT): Invierte un valor booleano (!true es false).
- **Lógicos:** Se usan para tomar decisiones. JS evalúa de izquierda a derecha.
 - && (AND): Devuelve true si **ambos** lados son verdaderos.
 - || (OR): Devuelve true si **al menos uno** de los lados es verdadero.
 - **Short-Circuit:** En A || B, si A es verdadero, JS ni siquiera mira B. Esto se usa para valores por defecto:
let nombre = inputUsuario || "Anónimo";
- **Comparación:**
 - > (mayor que), < (menor que), >= (mayor o igual), <= (menor o igual).
 - === (Igualdad Estricta): Comprueba si el valor **Y** el tipo son iguales. **Esta es la que debes usar siempre.**
 - == (Igualdad Débil): Intenta "convertir" los tipos antes de comparar. Es una fuente común de errores.

Anatomía del Código (Pre vs. Post Incremento)

```
let a = 5;
```

```
let b = 5;
```

```
// Post-incremento: usa el valor (5) y LUEGO lo incrementa (a=6)
```

```
console.log(a++); // Muestra: 5
```

```
// Pre-incremento: incrementa el valor (b=6) y LUEGO lo usa
```

```
console.log(++b); // Muestra: 6
```

Preguntas Frecuentes

¿Por qué $0.1 + 0.2$ no es igual a 0.3 ?

Las computadoras usan un sistema binario (base 2) para almacenar números. Los decimales como 0.1 y 0.2 no pueden representarse de forma exacta en binario, de la misma forma que $1/3$ no se puede escribir de forma exacta en decimal (0.333...). El resultado de la suma es algo como 0.30000000000000004. Es un problema común a casi todos los lenguajes de programación.

¿Por qué debería usar `const` en lugar de `let` si `let` me da más libertad para cambiar los valores?

La libertad total genera sistemas frágiles. Al usar `const` por defecto, le indicas a otros desarrolladores (y a tu yo del futuro) que ese valor fue diseñado para mantenerse intacto. Esto previene errores lógicos masivos, como sobrescribir accidentalmente el precio de un producto a mitad de una transacción. Usa `let` únicamente cuando estés seguro de que el valor debe mutar.

¿Qué sucede si intento sumar un `Number` y un `String`, por ejemplo $5 + "5"$?

Ocorre un fenómeno llamado **Coerción de Tipos (Type Coercion)**. El motor de JavaScript asume que quieres concatenar (unir) los textos. En lugar de ejecutar la operación matemática $5 + 5 = 10$, convierte el primer 5 a `String` y los une, dando como resultado el `String` "55". Este es uno de los errores más comunes al capturar datos de formularios HTML.

Ejemplo:

Vamos a simular el flujo de cálculo de un carrito de compras simple.

```
// 1. Declaración de variables estáticas (const)
const nombreCliente = "Ana Pérez";
const nombreProducto = "Monitor 24 pulgadas";
const precioUnitario = 200;
const impuestoIVA = 0.19; // 19%
const costoEnvio = 15;

// 2. Declaración de variables mutables (let)
let cantidadComprada = 2;
let subtotal;
let totalConImpuestos;

// 3. Operaciones Aritméticas
subtotal = precioUnitario * cantidadComprada;
const valorImpuesto = subtotal * impuestoIVA;
totalConImpuestos = subtotal + valorImpuesto + costoEnvio;

// 4. Operaciones con Strings (Concatenación)
const mensajeFinal = "Hola " + nombreCliente + ". El total de tu compra es: $" + totalConImpuestos;

// 5. Salida por consola para verificar
console.log("--- Resumen de Compra ---");
console.log("Producto:", nombreProducto);
console.log("Subtotal:", subtotal);
console.log("Total a pagar:", totalConImpuestos);
console.log("Mensaje al cliente:", mensajeFinal);
console.log("¿Es el total un número?", typeof totalConImpuestos);
```

Práctica acompañada

Vamos a construir el perfil básico de un usuario de una red social. Tienes el año de nacimiento y el año actual. Debes calcular la edad, crear el nombre completo y mostrar un mensaje de bienvenida.

Código de solución

```
// Paso 1: Crea dos constantes para el nombre y el apellido
const nombre = "Carlos";
const apellido = "Gómez";

// Paso 2: Crea una constante para el año de nacimiento y otra para el año actual
const anioNacimiento = 1995;
const anioActual = 2026;

// Paso 3: Crea una variable (tú decides si const o let) para calcular la edad
const edad = anioActual - anioNacimiento; // Usamos const porque este cálculo no mutará en esta ejecución

// Paso 4: Crea el nombre completo uniendo las variables de texto y un espacio en blanco
const nombreCompleto = nombre + " " + apellido;

// Paso 5: Muestra en consola: "El usuario [nombreCompleto] tiene [edad] años."
const mensajePerfil = "El usuario " + nombreCompleto + " tiene " + edad + " años.";
console.log(mensajePerfil);
```

Actividad de clase

Pair Programming

- **Driver (Piloto):** Toma el teclado, escribe el código, no diseña la lógica general.
- **Navigator (Copiloto):** Lee las instrucciones, revisa errores de sintaxis y dicta la lógica de negocio.

El Reto:

Deben construir una calculadora de propinas para un restaurante.

1. Declaren el valor total de la cuenta (ej. 50000).
2. Declaren el porcentaje de propina deseado (ej. 0.10 para 10%).
3. Calculen el valor exacto de la propina.
4. Calculen el gran total a pagar (cuenta + propina).
5. Impriman en consola usando `typeof` para asegurar que el gran total sigue siendo un `number` y no se convirtió accidentalmente en texto por mala concatenación.

Práctica Integradora

Ticket JIRA-101: Cálculo de Descuento Promocional

A. Instrucciones: El equipo de Marketing necesita un script que calcule el precio final de un artículo tras aplicarle un 15% de descuento fijo. Debes calcular cuánto dinero se descuenta y cuál es el precio final a cobrar.

B. Código Base:

```
const precioBase = 120000;  
const porcentajeDescuento = 0.15;  
let valorDescontado;  
let precioFinal;  
// Escribe tu lógica aquí
```

C. Solución Completa

```
const precioBase = 120000;  
const porcentajeDescuento = 0.15;  
  
const valorDescontado = precioBase * porcentajeDescuento;  
const precioFinal = precioBase - valorDescontado;  
  
console.log("Descuento aplicado: $" + valorDescontado);  
console.log("Precio final al cliente: $" + precioFinal);
```

Ticket JIRA-102: Formateo de Dirección de Envío

A. Instrucciones: Logística requiere que la dirección del cliente, la ciudad y el código postal se unan en una sola cadena de texto separada por comas para imprimir la etiqueta de la caja.

B. Código Base:

```
const calle = "Carrera 45 # 12-34";  
const ciudad = "Medellín";  
const codigoPostal = "050021";  
// Escribe tu lógica aquí
```

C. Solución Completa:

```
const calle = "Carrera 45 # 12-34";  
const ciudad = "Medellín";  
const codigoPostal = "050021";  
  
const etiquetaEnvio = calle + ", " + ciudad + ", " + codigoPostal;  
console.log("Etiqueta a imprimir: " + etiquetaEnvio);
```

Ticket JIRA-103: Validación Lógica de Inventario

A. Instrucciones: Ventas necesita saber si un producto se puede mostrar en la web. Solo se puede mostrar si tiene inventario mayor a cero (hasInventory) Y si el producto está marcado como activo (isActive). Usa operadores lógicos para obtener un booleano true o false.

B. Código Base:

```
const hasInventory = true;  
const isActive = false;  
// Escribe tu lógica aquí para determinar si es visible
```

C. Solución Completa:

```
const hasInventory = true;  
const isActive = false;  
  
const isVisibleInWeb = hasInventory && isActive;  
  
console.log("¿Mostrar en la tienda?: " + isVisibleInWeb); // Imprimirá false
```

Ejercicio Independiente

Ticket JIRA-205 - Módulo de Envíos: El cliente requiere calcular el costo de envío de un paquete. **Se te proporcionan tres variables:** una tarifa plana de envío, un recargo por peso extra y un recargo por distancia.

Debes crear un script desde cero que:

1. Declare las tres variables con valores numéricos a tu elección.
2. Declare una variable que sume los tres costos para obtener el costoTotalEnvio.
3. Cree una cadena de texto que diga: "El envío de tu paquete costará: \$ [costoTotalEnvio]".
4. Utiliza typeof en la consola para demostrar que tu costoTotalEnvio es un número y no se corrompió al hacer los cálculos.

Criterios de logro esperados:

- Uso correcto de const (no se deben usar let ni var para este caso estático).
- Correcta implementación de operadores aritméticos para la suma.
- Correcta concatenación de Strings con variables Number.
- Ausencia de errores de sintaxis en la consola de ejecución.

¿Debería Saber Algo Más?

Type Coercion (Coerción de Tipos): Este es el concepto "oculto" más importante. JavaScript es un lenguaje de "tipado débil", lo que significa que intenta "ayudarte" convirtiendo tipos automáticamente. A veces es útil, pero a menudo es confuso.

'5' - 1 da 4 (JavaScript convierte el string '5' a un Number).

'5' + 1 da '51' (El operador + prefiere la concatenación de strings).

'5' * '2' da 10 (Convierte ambos a Number).

Este comportamiento es la razón por la que **siempre usamos ===** y evitamos ==, para no ser víctimas de la coerción.

Precedencia de Operadores: Al igual que en matemáticas (PEMDAS), JavaScript tiene un orden estricto de operaciones. * y / se ejecutan antes que + y -. Los operadores lógicos también tienen un orden(! primero, luego &&, luego ||). Cuando tengas dudas, **usa paréntesis ()** para forzar el orden que deseas. El código claro siempre es mejor que el código "inteligente".

Strict Mode ("use strict"); Al inicio de sus archivos JavaScript, es una excelente práctica colocar el texto "use strict";. Esto fuerza al motor de JavaScript a ser mucho más severo con los errores silenciosos (por ejemplo, prohibirá usar una variable que no haya sido declarada previamente con let o const).